Forecasting On Air Passenger data

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

In [20]:
```
df=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Forecasting\AirPassenger
```

In [21]:
```
df.head()
```

Out[21]:

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

In [22]:
```
df.tail()
```

Out[22]:

	Month	Passengers
139	1960-08	606
140	1960-09	508
141	1960-10	461
142	1960-11	390
143	1960-12	432

In [25]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Month       144 non-null    datetime64[ns]
 1   Passengers  144 non-null    int64
dtypes: datetime64[ns](1), int64(1)
memory usage: 2.4 KB
```

In [26]:
```
df['Month'] = pd.to_datetime(df['Month'])
```

In [27]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Month       144 non-null    datetime64[ns]
 1   Passengers  144 non-null    int64
dtypes: datetime64[ns](1), int64(1)
memory usage: 2.4 KB
```

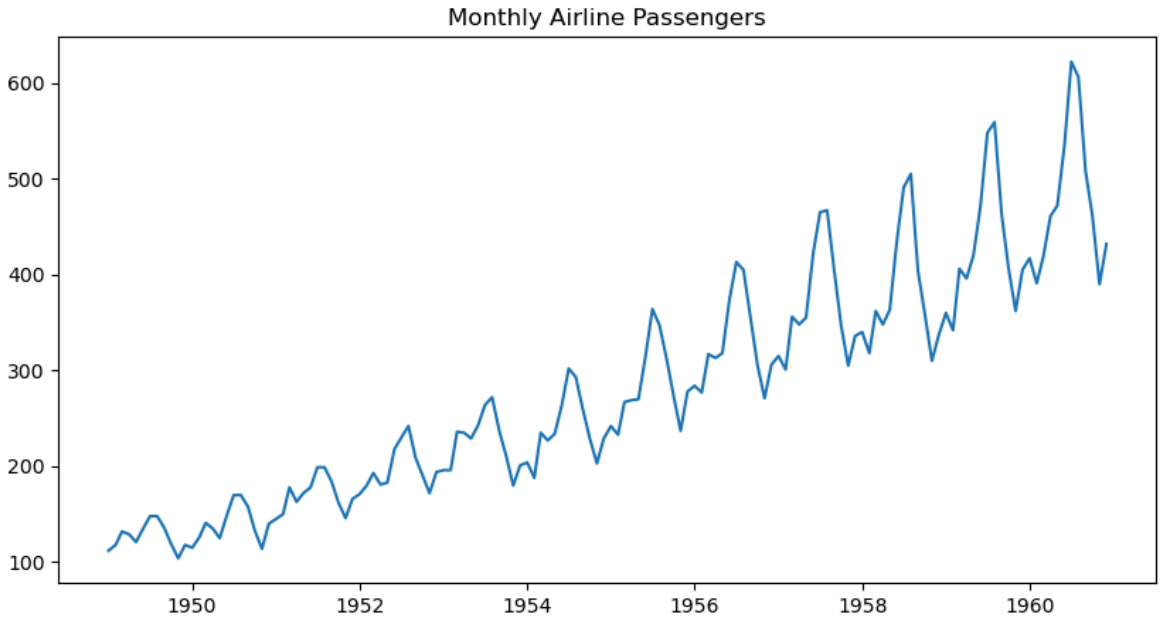
In [28]:
```
df.set_index('Month', inplace=True)
```

In [29]:
```
df.isnull().sum()
```

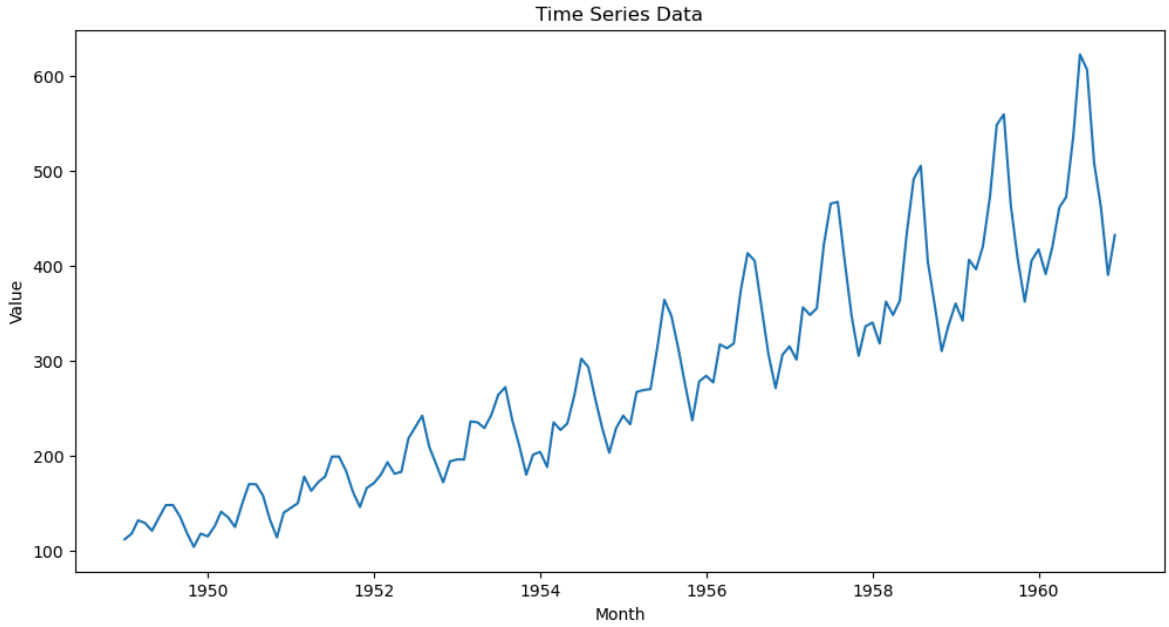
Out[29]:
```
Passengers    0
dtype: int64
```

In [30]:
```
df = df.dropna()
```

In [41]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(df)
plt.title("Monthly Airline Passengers")
plt.show()
```



In [31]:
```
plt.figure(figsize=(12,6))
plt.plot(df)
plt.title("Time Series Data")
plt.xlabel("Month")
plt.ylabel("Value")
plt.show()
```



In [32]:
```
result = adfuller(df['Passengers'])

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

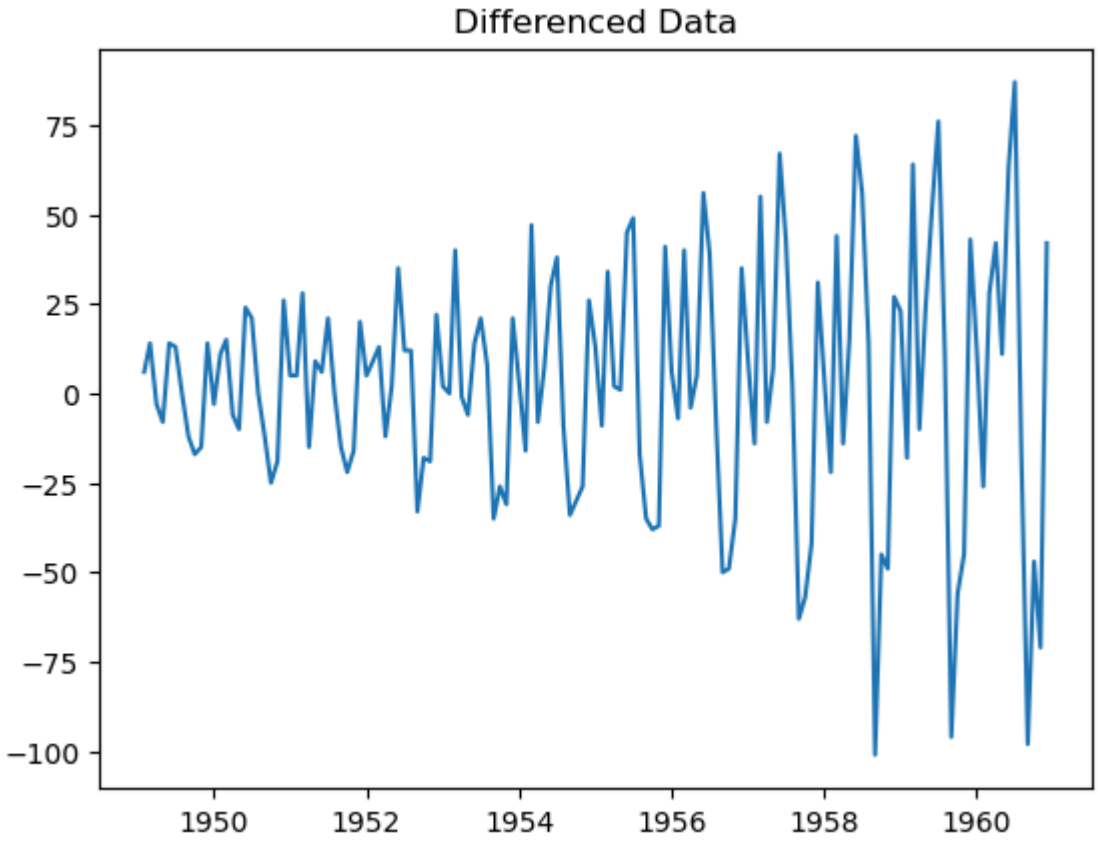
```
ADF Statistic: 0.8153688792060511
p-value: 0.991880243437641
```

Interpretation:

p < 0.05 → Stationary

p > 0.05 → Need differencing

In [33]:
```
df_diff = df.diff().dropna()

plt.plot(df_diff)
plt.title("Differenced Data")
plt.show()
```



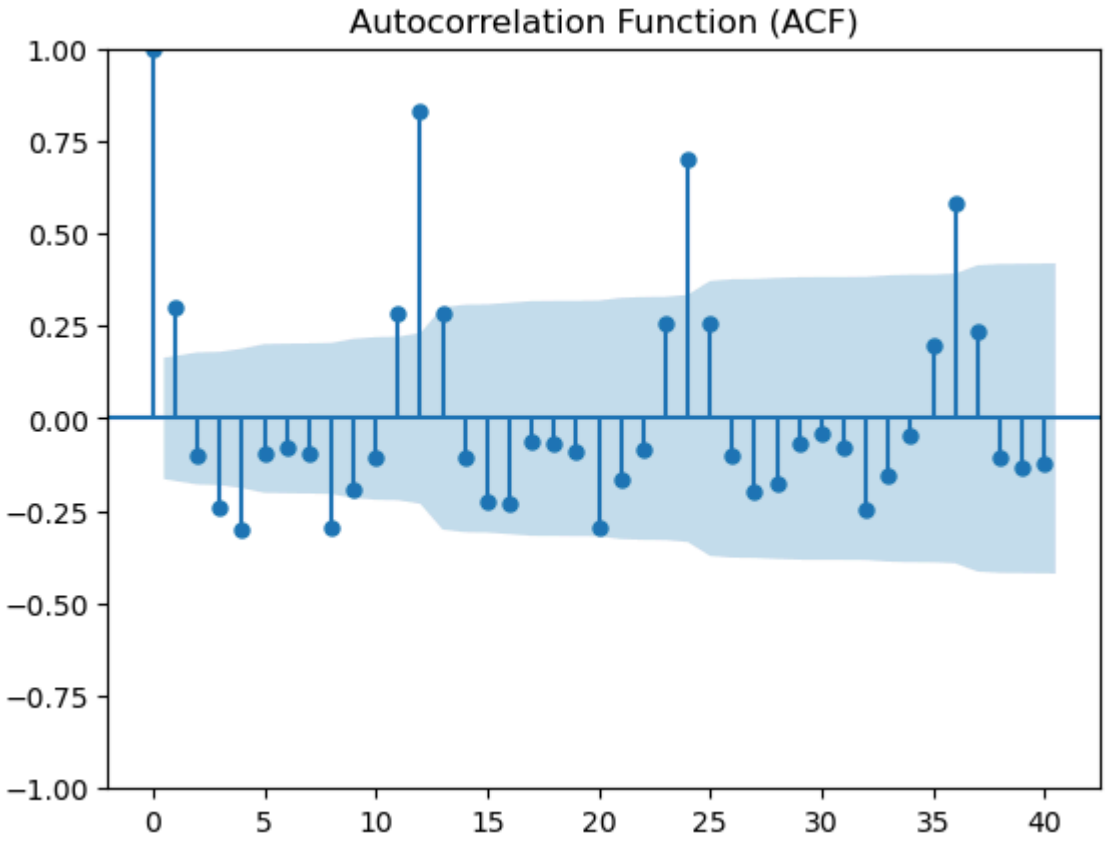
In [44]:
```
result = adfuller(df_diff['Passengers'])

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: -2.8292668241699874
p-value: 0.05421329028382711
```

In [42]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

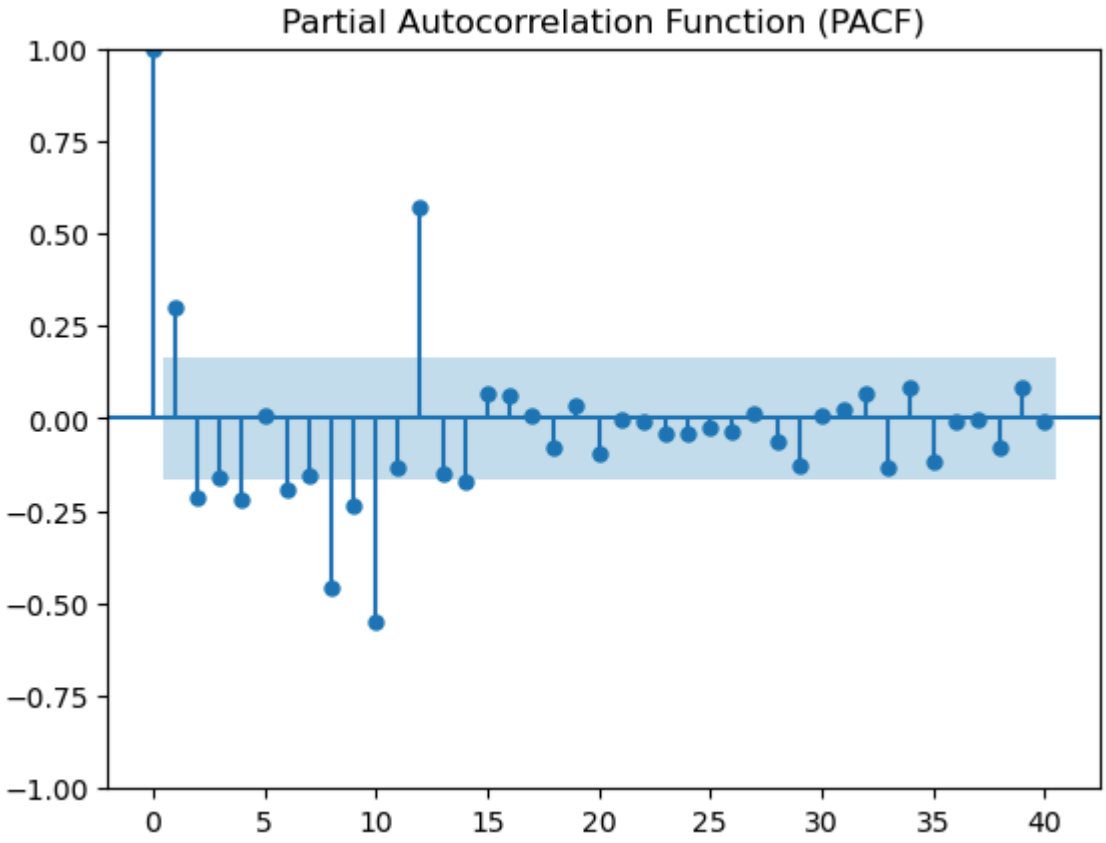
In [43]:
```
plt.figure(figsize=(10,5))

plot_acf(df_diff['Passengers'], lags=40)

plt.title("Autocorrelation Function (ACF)")
plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



In [45]:
```
plt.figure(figsize=(10,5))

plot_pacf(df_diff['Passengers'], lags=40)

plt.title("Partial Autocorrelation Function (PACF)")
plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



In [46]:
```
train_size = int(len(df) * 0.8)

train = df[:train_size]
test = df[train_size:]
```

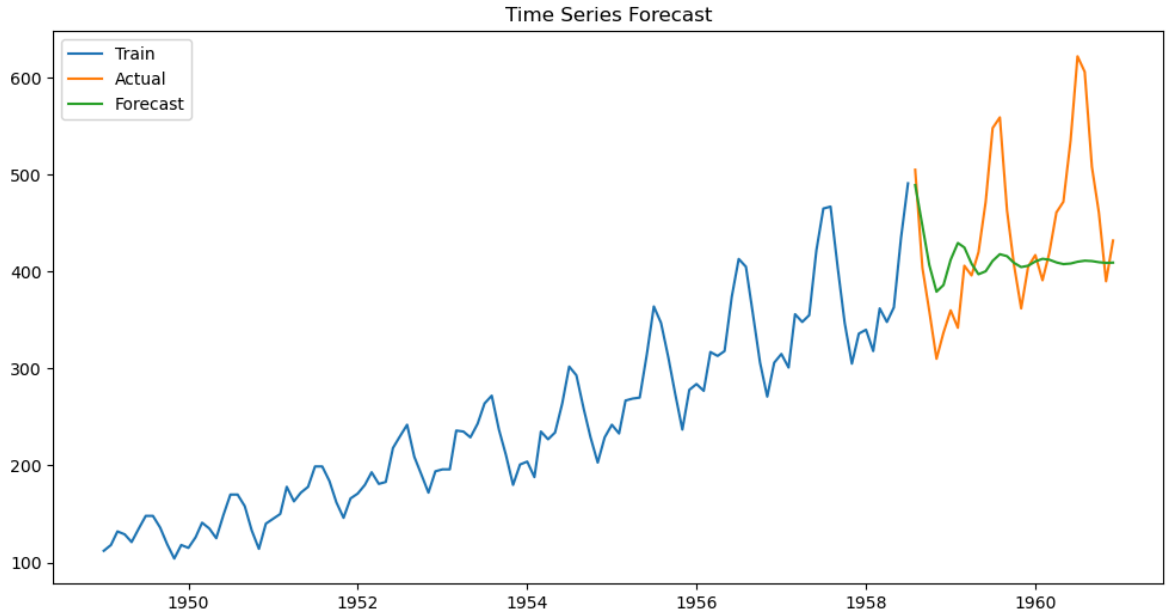
In [53]:
```
model = ARIMA(train['Passengers'], order=(2,1,5))
model_fit = model.fit()
print(model_fit.summary())
```

```
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:
978: UserWarning: Non-invertible starting MA parameters found. Using zeros as sta
rting parameters.
  warn('Non-invertible starting MA parameters found.'
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(2, 1, 5)   Log Likelihood                -511.860
Date:                Wed, 11 Mar 2026   AIC                           1039.720
Time:                        20:06:09   BIC                           1061.610
Sample:                    01-01-1949   HQIC                          1048.604
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.8558      0.101      8.451      0.000       0.657       1.054
ar.L2         -0.7418      0.073    -10.218      0.000      -0.884      -0.599
ma.L1         -0.5789      0.190     -3.042      0.002      -0.952      -0.206
ma.L2          0.3112      0.537      0.580      0.562      -0.740       1.363
ma.L3          0.4284      0.152      2.822      0.005       0.131       0.726
ma.L4         -0.6675      0.355     -1.882      0.060      -1.363       0.028
ma.L5         -0.0036      0.152     -0.024      0.981      -0.301       0.293
sigma2       437.9596    241.655      1.812      0.070     -35.675     911.594
===================================================================================
==
Ljung-Box (L1) (Q):                   0.26   Jarque-Bera (JB):                 3.
63
Prob(Q):                              0.61   Prob(JB):                         0.
16
Heteroskedasticity (H):               4.76   Skew:                            -0.
09
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.
14
===================================================================================
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-st
ep).
```

In [59]:
```
forecast = model_fit.forecast(steps=len(test))

forecast = pd.Series(forecast, index=test.index)
```

In [60]:
```
plt.figure(figsize=(12,6))

plt.plot(train, label="Train")
plt.plot(test, label="Actual")
plt.plot(forecast, label="Forecast")

plt.legend()
plt.title("Time Series Forecast")
plt.show()
```



In [61]:
```
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mean_squared_error(test, forecast))

print("MAE:", mae)
print("RMSE:", rmse)
```

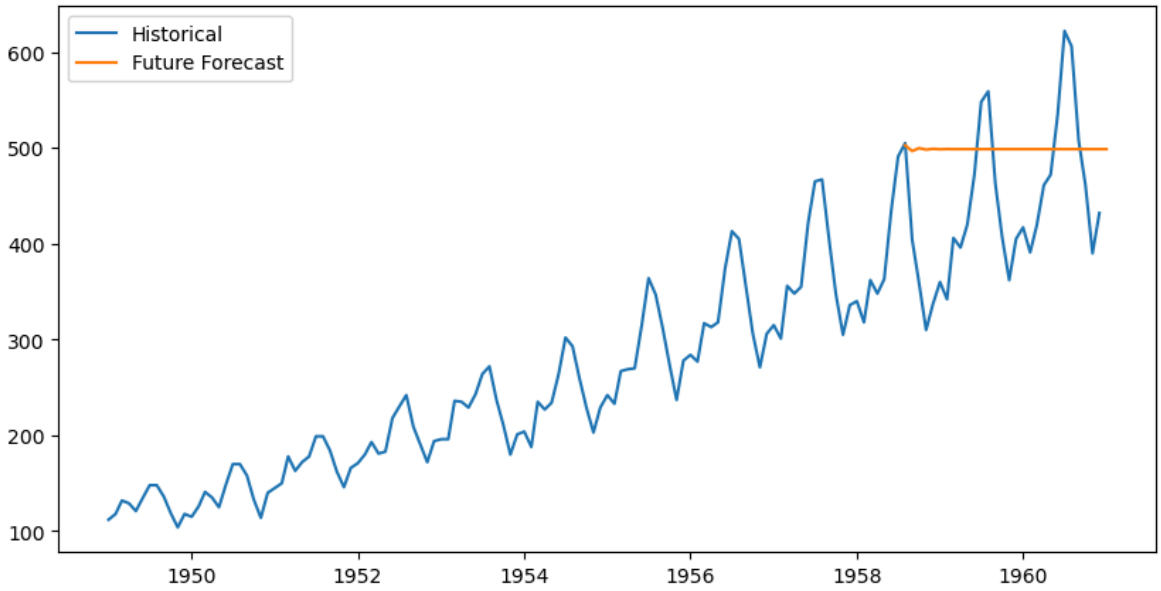
```
MAE: 59.867858163982234
RMSE: 80.96220237558704
```

In [57]:
```
future = model_fit.forecast(steps=30)

print(future)
```

```
1958-08-01    489.142350
1958-09-01    447.389308
1958-10-01    406.504181
1958-11-01    379.245500
1958-12-01    386.118997
1959-01-01    412.221181
1959-02-01    429.461914
1959-03-01    424.855476
1959-04-01    408.124509
1959-05-01    397.222398
1959-06-01    400.302421
1959-07-01    411.025244
1959-08-01    417.917596
1959-09-01    415.862506
1959-10-01    408.991165
1959-11-01    404.634801
1959-12-01    406.003386
1960-01-01    410.406080
1960-02-01    413.158905
1960-03-01    412.249110
1960-04-01    409.428517
1960-05-01    407.689400
1960-06-01    408.293218
1960-07-01    410.100007
1960-08-01    411.198436
1960-09-01    410.798298
1960-10-01    409.641067
1960-11-01    408.947472
1960-12-01    409.212262
1961-01-01    409.953365
Freq: MS, Name: predicted_mean, dtype: float64
```

In [40]:

```
plt.figure(figsize=(10,5))
plt.plot(df, label="Historical")
plt.plot(future, label="Future Forecast")
plt.legend()
plt.show()
```



Forecasting On WALLMART data

In [64]:
```
df_1=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Forecasting\train.csv'
```

In [65]:
```
df_1.head()
```

Out[65]:

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	2010-02-05	24924.50	False
1	1	1	2010-02-12	46039.49	True
2	1	1	2010-02-19	41595.55	False
3	1	1	2010-02-26	19403.54	False
4	1	1	2010-03-05	21827.90	False

In [67]:
```
df_1['Date'] = pd.to_datetime(df_1['Date'])
```

In [68]:
```
df_1 =df_1.sort_values('Date')
```

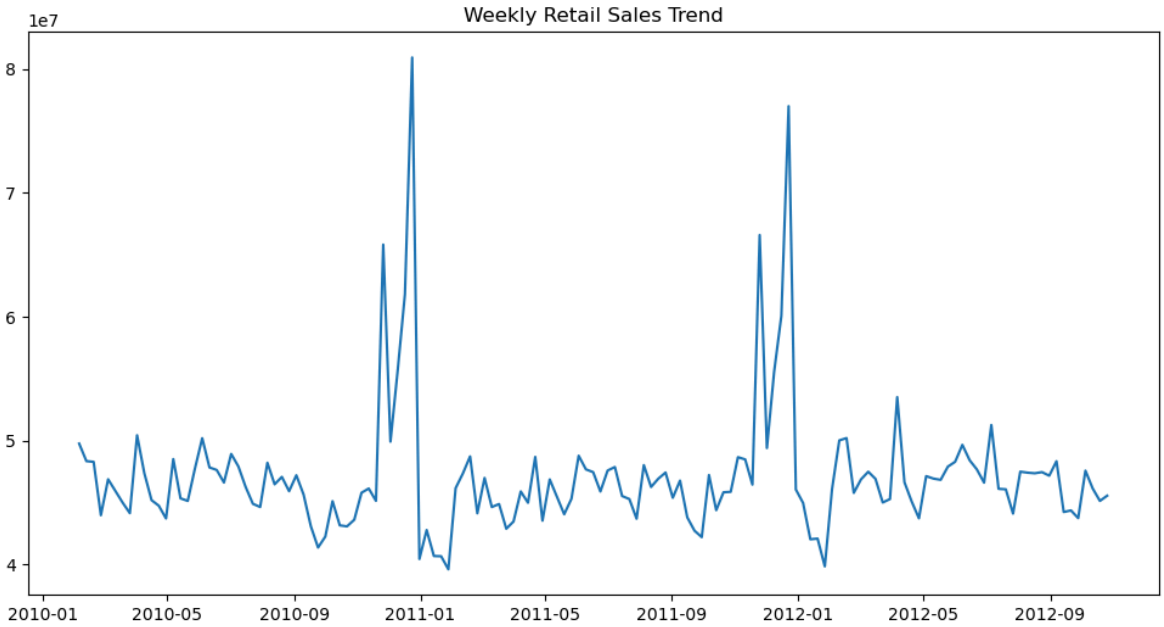
In [69]:
```
sales = df_1.groupby('Date')['Weekly_Sales'].sum().reset_index()
sales.head()
```

Out[69]:

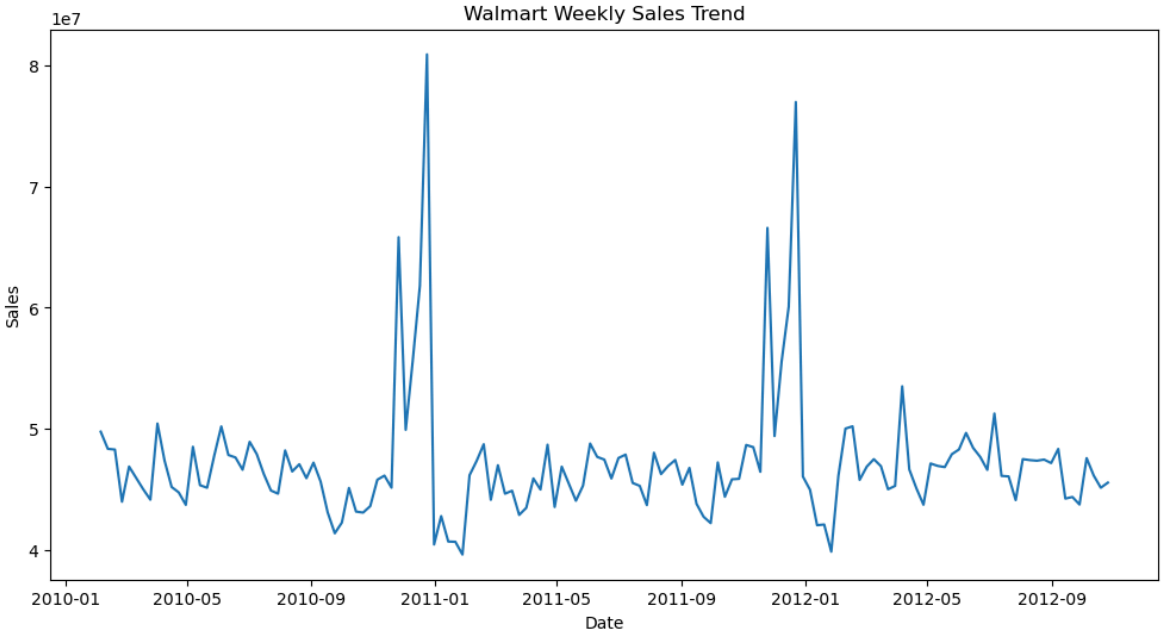
	Date	Weekly_Sales
0	2010-02-05	49750740.50
1	2010-02-12	48336677.63
2	2010-02-19	48276993.78
3	2010-02-26	43968571.13
4	2010-03-05	46871470.30

In [70]:
```
sales.set_index('Date', inplace=True)
```

In [82]:
```
plt.figure(figsize=(12,6))
plt.plot(sales)
plt.title("Weekly Retail Sales Trend")
plt.show()
```

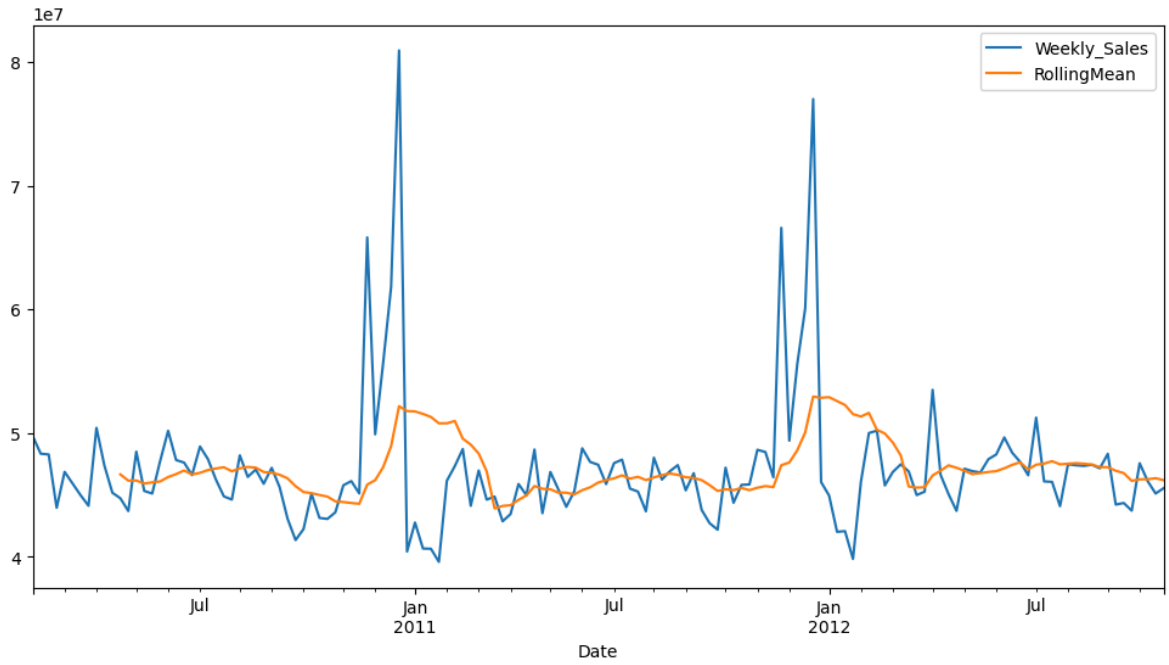


In [71]:
```
plt.figure(figsize=(12,6))
plt.plot(sales)
plt.title("Walmart Weekly Sales Trend")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()
```



In [83]:
```
sales['RollingMean'] = sales['Weekly_Sales'].rolling(window=12).mean()

sales[['Weekly_Sales','RollingMean']].plot(figsize=(12,6))
```

Out[83]: <Axes: xlabel='Date'>

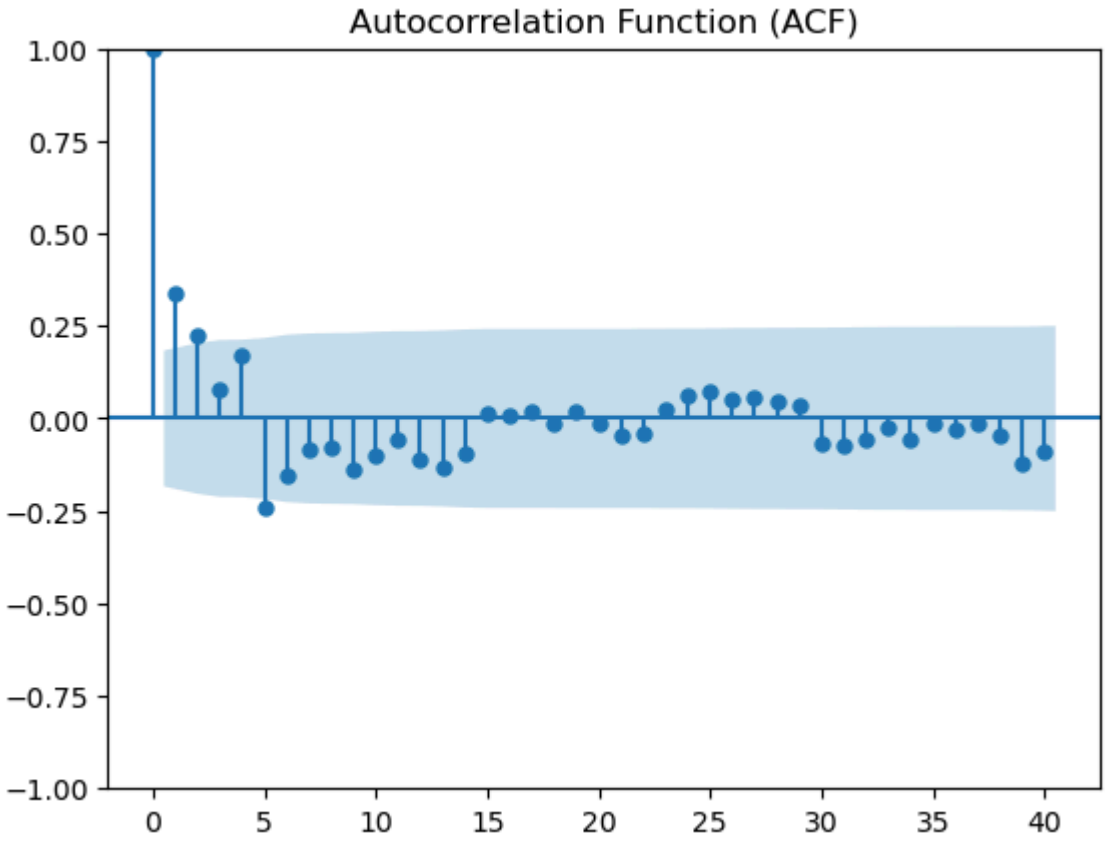


In [84]:
```
sales['lag_1'] = sales['Weekly_Sales'].shift(1)
sales['lag_2'] = sales['Weekly_Sales'].shift(2)
sales['lag_3'] = sales['Weekly_Sales'].shift(3)

sales.dropna(inplace=True)
```

In [72]:
```
train_size = int(len(sales)*0.8)

train = sales[:train_size]
test = sales[train_size:]
```

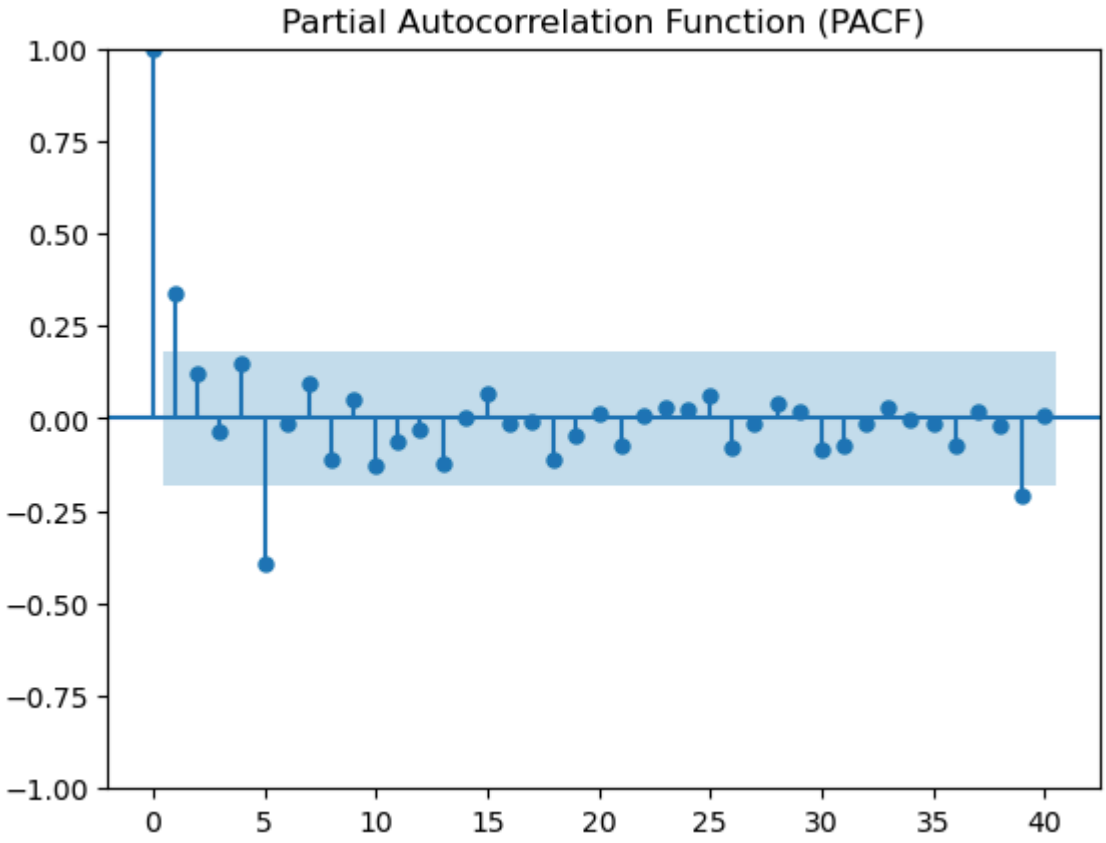
In [74]:
```
plt.figure(figsize=(10,5))

plot_acf(train['Weekly_Sales'], lags=40)

plt.title("Autocorrelation Function (ACF)")
plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



In [75]:
```
plt.figure(figsize=(10,5))

plot_pacf(train['Weekly_Sales'], lags=40)

plt.title("Partial Autocorrelation Function (PACF)")
plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



In [76]:
```
model = ARIMA(train['Weekly_Sales'], order=(5,1,0))
model_fit = model.fit()

print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:           Weekly_Sales   No. Observations:                  114
Model:                 ARIMA(5, 1, 0)   Log Likelihood               -1923.247
Date:                Wed, 11 Mar 2026   AIC                           3858.493
Time:                        20:36:48   BIC                           3874.858
Sample:                    02-05-2010   HQIC                          3865.134
                         - 04-06-2012
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.2956      0.051     -5.829      0.000      -0.395      -0.196
ar.L2         -0.2007      0.059     -3.395      0.001      -0.317      -0.085
ar.L3         -0.1744      0.073     -2.400      0.016      -0.317      -0.032
ar.L4          0.0952      0.072      1.315      0.188      -0.047       0.237
ar.L5         -0.1553      0.070     -2.204      0.028      -0.293      -0.017
sigma2      3.308e+13   3.95e-16   8.37e+28      0.000    3.31e+13    3.31e+13
===================================================================================
==
Ljung-Box (L1) (Q):                   2.59   Jarque-Bera (JB):               194.
61
Prob(Q):                              0.11   Prob(JB):                         0.
00
Heteroskedasticity (H):               3.67   Skew:                            -0.
32
Prob(H) (two-sided):                  0.00   Kurtosis:                         9.
40
===================================================================================
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-st
ep).
[2] Covariance matrix is singular or near-singular, with condition number 1.98e+4
4. Standard errors may be unstable.
```

```
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
```

In [91]:
```
pip install pmdarima
```

```
Collecting pmdarima
  Downloading pmdarima-2.1.1-cp313-cp313-win_amd64.whl.metadata (8.5 kB)
Requirement already satisfied: joblib>=0.11 in c:\users\anupa\anaconda3\lib\site-
packages (from pmdarima) (1.5.2)
Collecting Cython!=0.29.18,!=0.29.31,>=0.29 (from pmdarima)
  Downloading cython-3.2.4-cp313-cp313-win_amd64.whl.metadata (7.7 kB)
Requirement already satisfied: numpy>=1.21.6 in c:\users\anupa\anaconda3\lib\site
-packages (from pmdarima) (2.3.5)
Requirement already satisfied: pandas>=0.19 in c:\users\anupa\anaconda3\lib\site-
packages (from pmdarima) (2.3.3)
Requirement already satisfied: scikit-learn>=0.22 in c:\users\anupa\anaconda3\lib
\site-packages (from pmdarima) (1.7.2)
Requirement already satisfied: scipy>=1.13.0 in c:\users\anupa\anaconda3\lib\site
-packages (from pmdarima) (1.16.3)
Requirement already satisfied: statsmodels>=0.14.5 in c:\users\anupa\anaconda3\li
b\site-packages (from pmdarima) (0.14.5)
Requirement already satisfied: urllib3 in c:\users\anupa\anaconda3\lib\site-packa
ges (from pmdarima) (2.5.0)
Requirement already satisfied: setuptools!=50.0.0,>=42 in c:\users\anupa\anaconda
3\lib\site-packages (from pmdarima) (80.9.0)
Requirement already satisfied: packaging>=17.1 in c:\users\anupa\anaconda3\lib\si
te-packages (from pmdarima) (25.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\anupa\anaconda3
\lib\site-packages (from pandas>=0.19->pmdarima) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\anupa\anaconda3\lib\site-
packages (from pandas>=0.19->pmdarima) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\anupa\anaconda3\lib\sit
e-packages (from pandas>=0.19->pmdarima) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\anupa\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\anupa\anaconda3\l
ib\site-packages (from scikit-learn>=0.22->pmdarima) (3.5.0)
Requirement already satisfied: patsy>=0.5.6 in c:\users\anupa\anaconda3\lib\site-
packages (from statsmodels>=0.14.5->pmdarima) (1.0.1)
Downloading pmdarima-2.1.1-cp313-cp313-win_amd64.whl (711 kB)
   ---------------------------------------- 0.0/711.9 kB ? eta -:--:--
   ---------------------------------------- 0.0/711.9 kB ? eta -:--:--
   -------------- ------------------------- 262.1/711.9 kB ? eta -:--:--
   ------------------------------ ---------- 524.3/711.9 kB 1.2 MB/s eta 0:00:01
   ---------------------------------------- 711.9/711.9 kB 1.2 MB/s  0:00:00
Downloading cython-3.2.4-cp313-cp313-win_amd64.whl (2.8 MB)
   ---------------------------------------- 0.0/2.8 MB ? eta -:--:--
   --- ------------------------------------ 0.3/2.8 MB ? eta -:--:--
   ------- -------------------------------- 0.5/2.8 MB 1.8 MB/s eta 0:00:02
   --------------- ------------------------ 1.0/2.8 MB 2.0 MB/s eta 0:00:01
   ---------------------- ----------------- 1.6/2.8 MB 2.2 MB/s eta 0:00:01
   ------------------------------ --------- 2.1/2.8 MB 2.4 MB/s eta 0:00:01
   ---------------------------------------- 2.8/2.8 MB 2.4 MB/s  0:00:01
Installing collected packages: Cython, pmdarima

   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   ---------------------------------------- 0/2 [Cython]
   -------------------- ------------------- 1/2 [pmdarima]
   -------------------- ------------------- 1/2 [pmdarima]
```

```
   -------------------- ------------------- 1/2 [pmdarima]
   ---------------------------------------- 2/2 [pmdarima]

Successfully installed Cython-3.2.4 pmdarima-2.1.1
Note: you may need to restart the kernel to use updated packages.
```

In [92]:
```
from pmdarima import auto_arima

model = auto_arima(train,
                   seasonal=True,
                   m=12,
                   trace=True)
```

```
Performing stepwise search to minimize aic
 ARIMA(2,0,2)(1,0,1)[12] intercept   : AIC=inf, Time=0.85 sec
 ARIMA(0,0,0)(0,0,0)[12] intercept   : AIC=3886.296, Time=0.02 sec
 ARIMA(1,0,0)(1,0,0)[12] intercept   : AIC=3876.060, Time=0.21 sec
 ARIMA(0,0,1)(0,0,1)[12] intercept   : AIC=3879.935, Time=0.06 sec
 ARIMA(0,0,0)(0,0,0)[12]             : AIC=4356.183, Time=0.02 sec
 ARIMA(1,0,0)(0,0,0)[12] intercept   : AIC=3874.374, Time=0.03 sec
 ARIMA(1,0,0)(0,0,1)[12] intercept   : AIC=3875.883, Time=0.07 sec
 ARIMA(1,0,0)(1,0,1)[12] intercept   : AIC=inf, Time=0.41 sec
 ARIMA(2,0,0)(0,0,0)[12] intercept   : AIC=3874.676, Time=0.03 sec
 ARIMA(1,0,1)(0,0,0)[12] intercept   : AIC=3875.641, Time=0.03 sec
 ARIMA(0,0,1)(0,0,0)[12] intercept   : AIC=3878.647, Time=0.03 sec
 ARIMA(2,0,1)(0,0,0)[12] intercept   : AIC=3876.670, Time=0.07 sec
 ARIMA(1,0,0)(0,0,0)[12]             : AIC=inf, Time=0.02 sec

Best model:  ARIMA(1,0,0)(0,0,0)[12] intercept
Total fit time: 1.863 seconds
```

In [ ]:
```
model
```

In [77]:
```
forecast = model_fit.forecast(steps=len(test))

forecast = pd.Series(forecast, index=test.index)
```

In [78]:
```
plt.figure(figsize=(12,6))

plt.plot(train, label="Train")
plt.plot(test, label="Actual")
plt.plot(forecast, label="Forecast")

plt.legend()
plt.title("Walmart Sales Forecast")
plt.show()
```

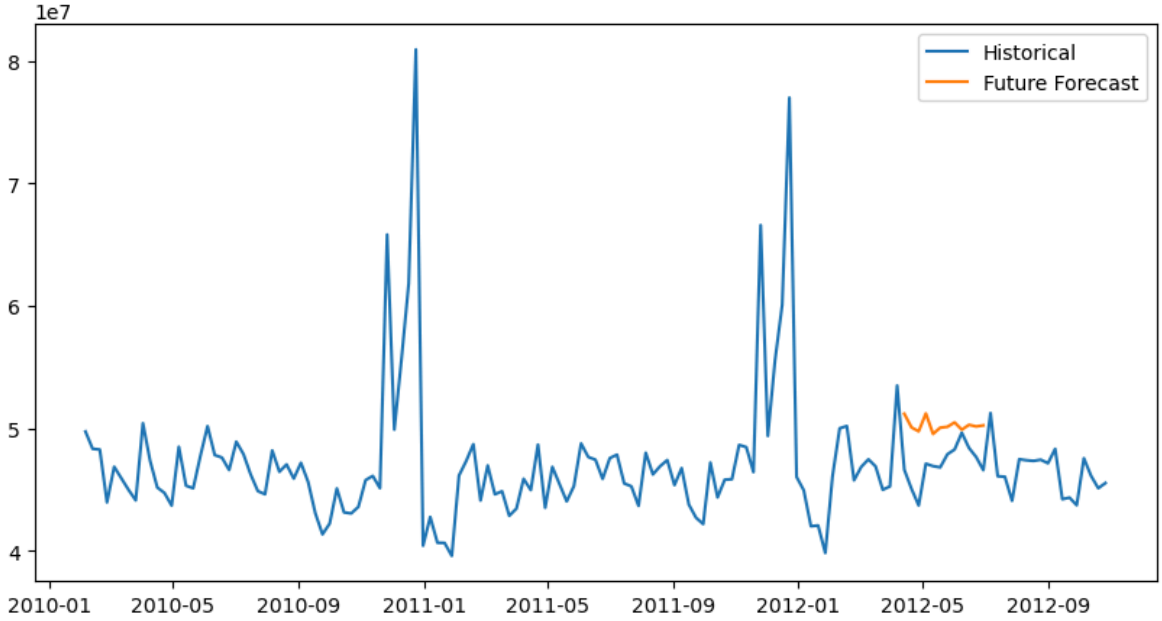
In [79]:
```
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mean_squared_error(test, forecast))

print("MAE:", mae)
print("RMSE:", rmse)
```

```
MAE: 3597509.817290272
RMSE: 3924134.1178595982
```

In [80]:
```
future = model_fit.forecast(steps=12)

print(future)
```

```
2012-04-13    5.119490e+07
2012-04-20    5.008505e+07
2012-04-27    4.976354e+07
2012-05-04    5.122423e+07
2012-05-11    4.955298e+07
2012-05-18    5.006257e+07
2012-05-25    5.013423e+07
2012-06-01    5.049133e+07
2012-06-08    4.989652e+07
2012-06-15    5.029623e+07
2012-06-22    5.016284e+07
2012-06-29    5.024870e+07
Freq: W-FRI, Name: predicted_mean, dtype: float64
```

In [81]:
```
plt.figure(figsize=(10,5))
plt.plot(sales, label="Historical")
plt.plot(future, label="Future Forecast")
plt.legend()
plt.show()
```



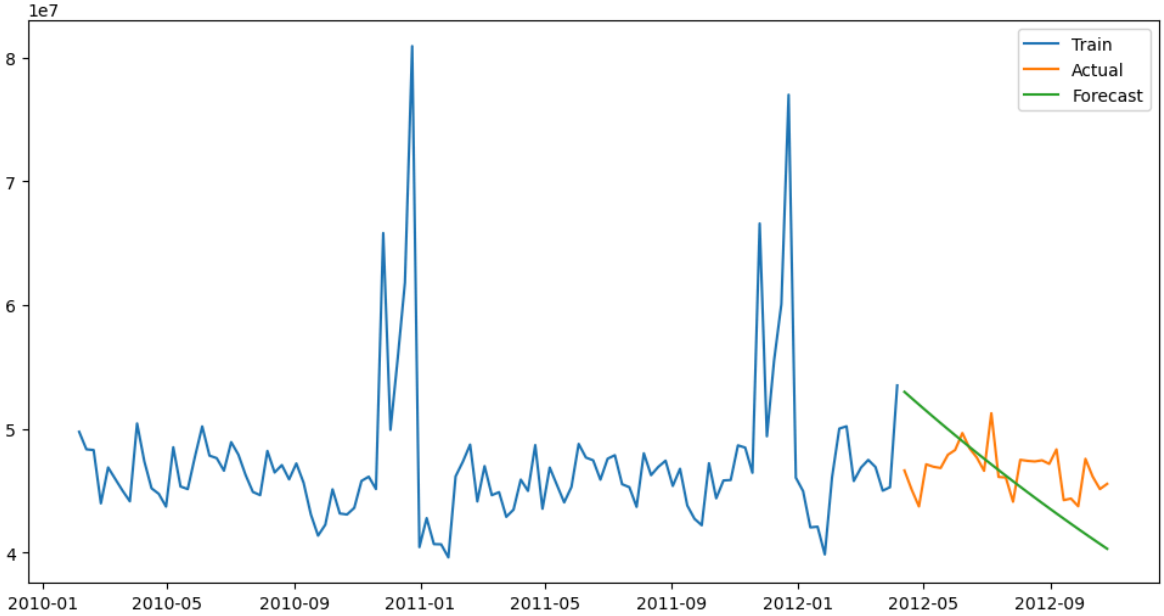
In [87]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.statespace.sarimax import SARIMAX

#from prophet import Prophet
```

SARIMA Model

In [93]:
```
#model = SARIMAX(train['Weekly_Sales'],
                #order=(1,1,1),
                #seasonal_order=(1,1,1,12))
model = SARIMAX(train['Weekly_Sales'],
                order=(1,0,0),
                seasonal_order=(0,0,0,12))
result = model.fit()

forecast = result.forecast(steps=len(test))
```

```
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
```

In [94]:
```
plt.figure(figsize=(12,6))

plt.plot(train.index, train['Weekly_Sales'], label='Train')
plt.plot(test.index, test['Weekly_Sales'], label='Actual')
plt.plot(test.index, forecast, label='Forecast')

plt.legend()
plt.show()
```



SARIMAX ON WALLMART DATA

In [4]:
```
train= pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Forecasting\train.cs
features = pd.read_csv(r"C:\Users\anupa\Desktop\Python Learning\Forecasting\feat
stores = pd.read_csv(r"C:\Users\anupa\Desktop\Python Learning\Forecasting\stores
```

In [5]:
```
# Merge datasets
df = train.merge(features, on=["Store", "Date", "IsHoliday"])
df = df.merge(stores, on="Store")

df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')

df.head()
```

Out[5]:

	Store	Dept	Date	Weekly_Sales	IsHoliday	Temperature	Fuel_Price	MarkDo
0	1	1	2010-02-05	24924.50	False	42.31	2.572	
277665	29	5	2010-02-05	15552.08	False	24.36	2.788	
277808	29	6	2010-02-05	3200.22	False	24.36	2.788	
277951	29	7	2010-02-05	10820.05	False	24.36	2.788	
278094	29	8	2010-02-05	20055.64	False	24.36	2.788	

In [6]:
```
#Aggregate Weekly Sales
#For simplicity we forecast total weekly sales.
```

```
sales = df.groupby('Date')['Weekly_Sales'].sum().reset_index()
sales = sales.set_index('Date')
```

In [7]:
```
#Add Exogenous Variables
exog = df.groupby('Date')[['Temperature','Fuel_Price','CPI','Unemployment']].mea
```

In [8]:
```
#Train Test Split
train_size = int(len(sales)*0.8)

train_sales = sales[:train_size]
test_sales = sales[train_size:]

train_exog = exog[:train_size]
test_exog = exog[train_size:]
```

In [10]:
```
#Build SARIMAX Model
from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(train_sales,
                exog=train_exog,
                order=(1,1,1),
                seasonal_order=(1,1,1,52))

results = model.fit()

print(results.summary())
```

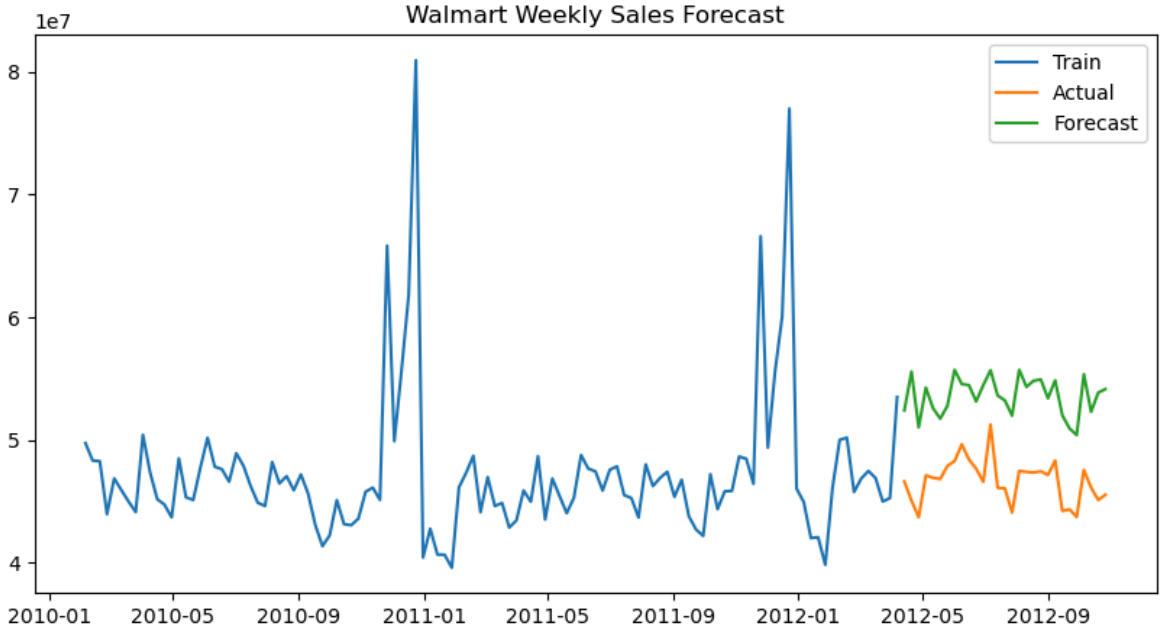
```
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency W-FRI
will be used.
  self._init_dates(dates, freq)
C:\Users\anupa\anaconda3\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:
866: UserWarning: Too few observations to estimate starting parameters for season
al ARMA. All parameters except for variances will be set to zeros.
  warn('Too few observations to estimate starting parameters%s.'
```

```
                                     SARIMAX Results
==============================================================================
=========
Dep. Variable:                      Weekly_Sales   No. Observations:
114
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 52)   Log Likelihood
-972.262
Date:                            Thu, 12 Mar 2026   AIC
1962.523
Time:                                    08:16:02   BIC
1981.521
Sample:                                02-05-2010   HQIC
1969.969
                                     - 04-06-2012
Covariance Type:                              opg
==============================================================================
                   coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Temperature   5.414e+04   7.67e+04      0.706      0.480   -9.61e+04    2.04e+05
Fuel_Price    3.806e+04   6503.214      5.853      0.000    2.53e+04    5.08e+04
CPI           7.164e+04   4.14e+05      0.173      0.863    -7.4e+05    8.83e+05
Unemployment -6.023e+06   2.21e+04   -272.170      0.000   -6.07e+06   -5.98e+06
ar.L1            0.1859      3.090      0.060      0.952      -5.870       6.241
ma.L1           -0.2326      3.074     -0.076      0.940      -6.258       5.792
ar.S.L52         0.7336      0.136      5.402      0.000       0.467       1.000
ma.S.L52        -0.9419      0.283     -3.329      0.001      -1.496      -0.387
sigma2        4.234e+12      0.082   5.18e+13      0.000    4.23e+12    4.23e+12
===================================================================================
==
Ljung-Box (L1) (Q):                   6.99   Jarque-Bera (JB):                 4.
81
Prob(Q):                              0.01   Prob(JB):                         0.
09
Heteroskedasticity (H):               1.36   Skew:                             0.
44
Prob(H) (two-sided):                  0.50   Kurtosis:                         4.
05
===================================================================================
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-st
ep).
[2] Covariance matrix is singular or near-singular, with condition number 5.84e+2
8. Standard errors may be unstable.
```

In [11]:
```
forecast = results.predict(start=len(train_sales),
                           end=len(sales)-1,
                           exog=test_exog)
```

In [12]:
```
#Plot Forecast vs Actual
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

plt.plot(train_sales, label="Train")
plt.plot(test_sales, label="Actual")
plt.plot(forecast, label="Forecast")

plt.legend()
```

```
plt.title("Walmart Weekly Sales Forecast")
plt.show()
```



In [13]:
```
#Evaluate Forecast
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(test_sales, forecast)

print("MAE:", mae)
```

```
MAE: 6904991.032679149
```

In [ ]: